

Animal Classification

Project Report

Prepared by:

Sadman Sakib Khan Promit

Institution:

North South University

Course:

Pattern Recognition

Semester:

May 2023

1. Introduction

The project compared the effects of two regularization techniques: batch normalization and dropout. For this task, ten given hyperparameter configurations were explored.

They are given below:

(Dropout, BN) = {(0.0, False), (0.0, True), (0.2, False), (0.2, True), (0.3, False), (0.3, True), (0.4, False), (0.4, True), (0.5, False), (0.5, True)}

The critical points of this project are given below:

- Benchmarking Regularizations
- Explored both with and without transfer learning.
- For transfer learning, ResNet152 architecture was used.
- For non-transfer learning Squeeze-and-Excitation with ResNet152 architecture was used.
- Tuned dropout probability and batch normalization, trained ten models for each architecture, producing twenty models.
- The pre-trained model was frozen; only FC optimized in the transfer learning model.
- Reproducible results, i.e., deterministic code, random seed: 42
- Batch normalization didn't perform well in the transfer learning models.
- The transfer learning model outperformed the non-transfer learning model by a wide margin.
- The best validation accuracy reached 97.50%.

2. Dataset

Five different public datasets available at Kaggle were combined randomly for the dataset. Each of the twenty models was trained on this custom dataset.

Dataset's statistics are given below:

- Size: 1.37 GB
- Number of samples: 5000
- Number of classes: 20
- Split Ratio: 80:20 (Stratify)
- Classes: {buffalo, butterfly, cat, crab, eagle, elephant, fish, giraffe, jellyfish, lion, monkey, penguin, polar_bear, sea_turtle, shark, spider, starfish, tiger, whale, zebra}

The Kaggle dataset's links are given below:

- [African Wildlife](#)
- [Animal Image Dataset\(DOG, CAT and PANDA\)](#)
- [Animals Detection Images Dataset](#)
- [Animals-10](#)
- [Wildlife Animals Images](#)



Figure 1: Snapshot of the dataset

3. Hardware

The Asus 2021 ROG Strix G15 G513IE laptop was used for training and inferences.

The hardware configurations are given below:

- CPU: AMD Ryzen™ 7 4800H (8 Cores, 16 Threads, Max 4.2 GHz)
- GPU: RTX 3050Ti (4 GB VRAM)
- Memory: 32 GB

Estimated hardware usage during training time is given below:

Model	CPU (Utilization % @ GHz)	GPU (GB)	Memory (GB)
ResNet	73, 4.17	1.5	11.2
SeNet	16, 4.00	3.3	22

Estimated hardware usage during inference time is given below:

Model	CPU (Utilization % @ GHz)	GPU (GB)	Memory (GB)
ResNet	66, 4.09	1.5	11.4
SeNet	14, 3.93	3.3	21.9

Screenshots of hardware usage during training and inference times are given below:



Figure 2: Screenshot of hardware usage during ResNet training.

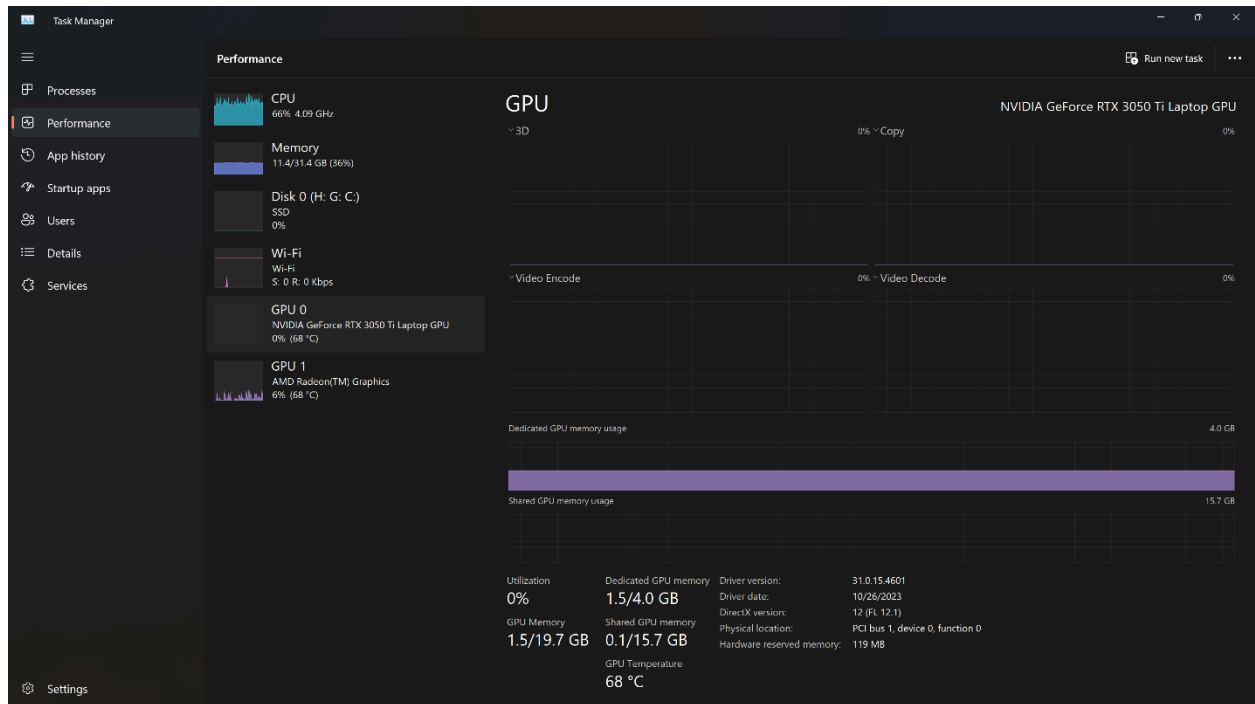


Figure 3: Screenshot of hardware usage during ResNet inference.

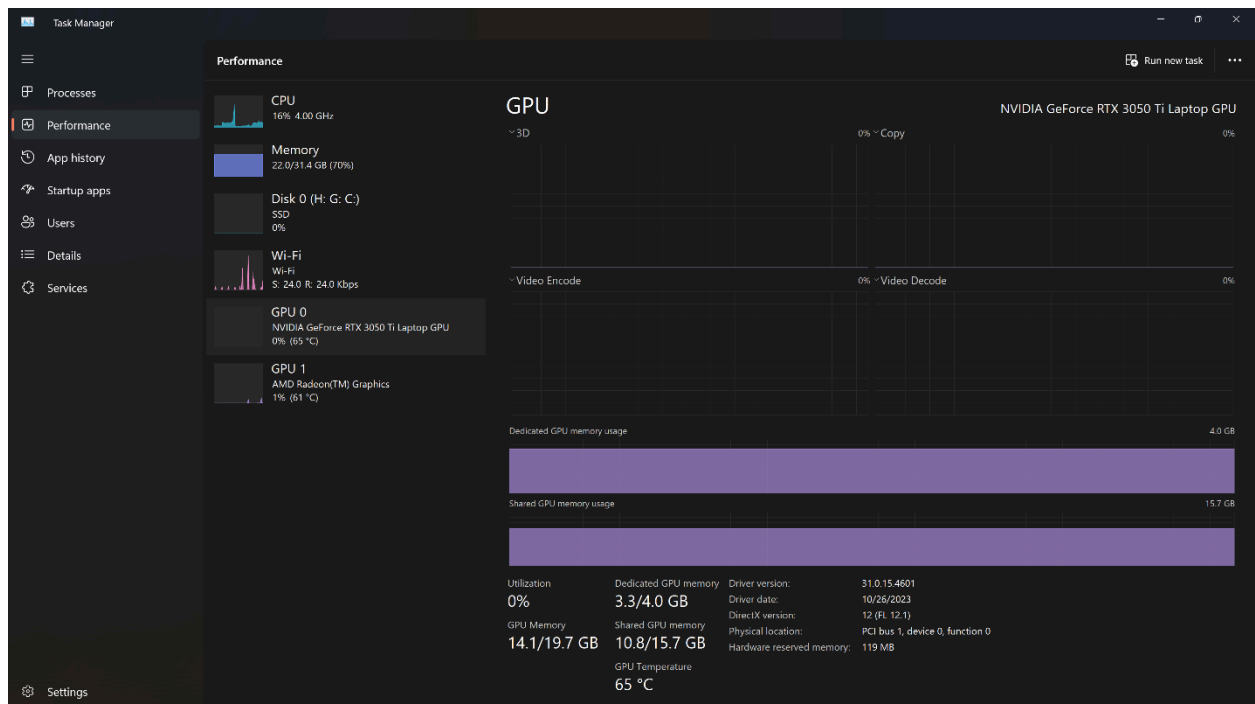


Figure 4: Screenshot of hardware usage during SeNet training.



Figure 5: Screenshot of hardware usage during SeNet inference.

4. Models

The following pre-trained weights from PyTorch have been initialized for the ResNet model:

```
weights=models.ResNet152_Weights.IMAGENET1K_V2
```

It is important to note that the entire network, including the FC layers, is considered for calculating the parameters and computations. As the classes 20 are used instead of the default 1000, the parameter count is less than the default. Also, it is to be noted that, as these numbers are for a forward pass, frozen parameters are also included in the transfer learning model. For simplicity, a batch size of 1 is used for the number of calculations instead of 64. As no library functions can find this from PyTorch, a custom function was defined, calculating the floating-point operations (FLOPS), where only basic operations like addition and multiplication are considered. Batch normalization and activations are not included for simplicity.

The following numbers of parameters and computations have been found from PyTorch in each model:

Model	# of parameters	# of computations
ResNet	58188884	3639319461888
SeNet	64755284	901191244447744

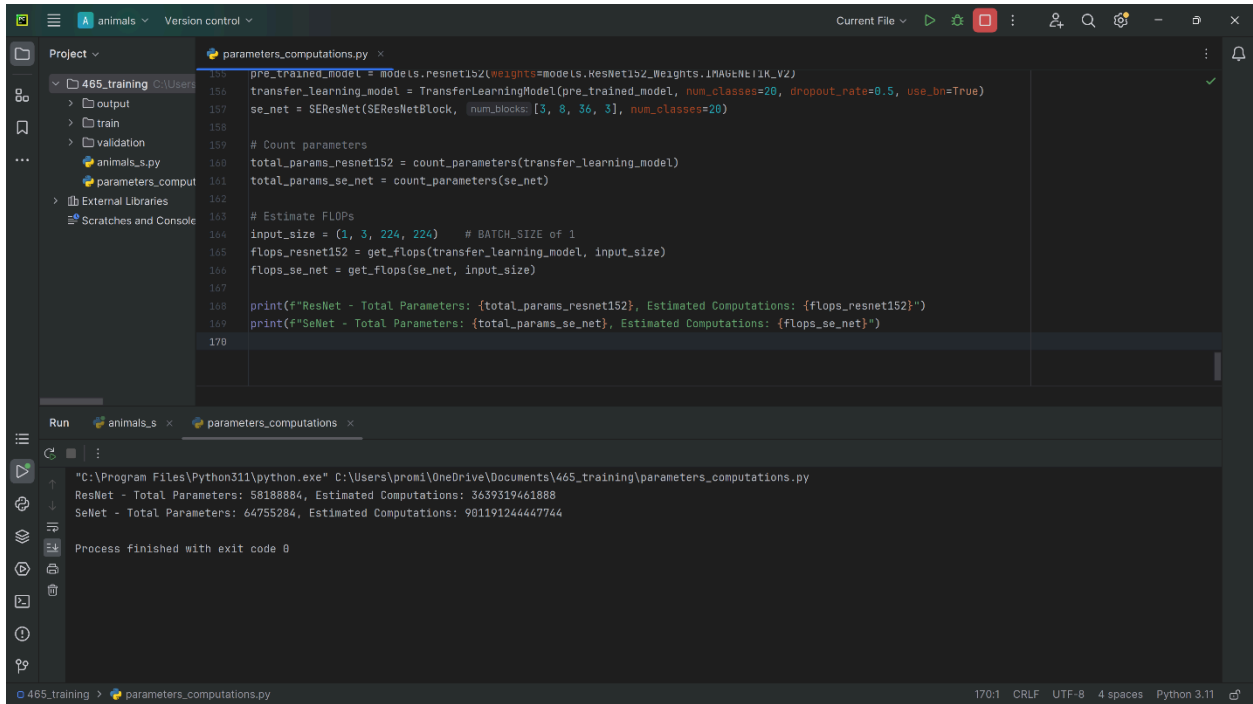


Figure 6: Screenshot of the number of parameters and computations calculation.

The code used to calculate the given values is given below:

```

import torch
import torch.nn as nn
import torchvision.models as models

class TransferLearningModel(nn.Module):
    def __init__(self, base_model, num_classes, dropout_rate, use_bn=False):
        super(TransferLearningModel, self).__init__()
        self.base_model = base_model

        # Freeze the parameters of the pre-trained model
        for param in self.base_model.parameters():
            param.requires_grad = False

        # Replacing the last fully connected layer with a custom classifier
        num_features = self.base_model.fc.in_features # Pretrained model
        features
        self.base_model.fc = self._build_classifier(num_features, num_classes,
        dropout_rate, use_bn)

        @staticmethod
        def _build_classifier(in_features, num_classes, dropout_rate, use_bn): #
        FC Layers
        layers = [nn.Dropout(dropout_rate)] # DP Layer
        if use_bn: # BN Layer
            layers.append(nn.BatchNorm1d(in_features))
        layers.append(nn.Linear(in_features, num_classes)) # Dense Layer
        return nn.Sequential(*layers)

```

```

def forward(self, x):
    return self.base_model(x)

# Custom SELayer definition
class SELayer(nn.Module):
    def __init__(self, channel, reduction=16):
        super(SELayer, self).__init__()
        self.avg_pool = nn.AdaptiveAvgPool2d(1)
        self.fc = nn.Sequential(
            nn.Linear(channel, channel // reduction, bias=False),
            nn.ReLU(inplace=True),
            nn.Linear(channel // reduction, channel, bias=False),
            nn.Sigmoid()
        )

    def forward(self, x):
        b, c, _, _ = x.size()
        y = self.avg_pool(x).view(b, c)
        y = self.fc(y).view(b, c, 1, 1)
        return x * y.expand_as(x)

# Custom SEResNetBlock definition
class SEResNetBlock(nn.Module):
    expansion = 4

    def __init__(self, in_planes, planes, stride=1, reduction=16):
        super(SEResNetBlock, self).__init__()
        self.conv1 = nn.Conv2d(in_planes, planes, kernel_size=1, bias=False)
        self.bn1 = nn.BatchNorm2d(planes)
        self.conv2 = nn.Conv2d(planes, planes, kernel_size=3, stride=stride,
padding=1, bias=False)
        self.bn2 = nn.BatchNorm2d(planes)
        self.conv3 = nn.Conv2d(planes, self.expansion * planes, kernel_size=1,
bias=False)
        self.bn3 = nn.BatchNorm2d(self.expansion * planes)

        self.se = SELayer(self.expansion * planes, reduction)

        self.shortcut = nn.Sequential()
        if stride != 1 or in_planes != self.expansion * planes:
            self.shortcut = nn.Sequential(
                nn.Conv2d(in_planes, self.expansion * planes, kernel_size=1,
stride=stride, bias=False),
                nn.BatchNorm2d(self.expansion * planes)
            )

    def forward(self, x):
        out = nn.ReLU()(self.bn1(self.conv1(x)))
        out = nn.ReLU()(self.bn2(self.conv2(out)))
        out = self.bn3(self.conv3(out))
        out = self.se(out)
        out += self.shortcut(x)
        out = nn.ReLU()(out)
        return out

```



```

# Custom SEResNet model definition
class SEResNet(nn.Module):
    def __init__(self, block, num_blocks, num_classes=1000):
        super(SEResNet, self).__init__()
        self.in_planes = 64

        self.conv1 = nn.Conv2d(3, 64, kernel_size=3, stride=1, padding=1,
bias=False)
        self.bn1 = nn.BatchNorm2d(64)
        self.layer1 = self._make_layer(block, 64, num_blocks[0], stride=1)
        self.layer2 = self._make_layer(block, 128, num_blocks[1], stride=2)
        self.layer3 = self._make_layer(block, 256, num_blocks[2], stride=2)
        self.layer4 = self._make_layer(block, 512, num_blocks[3], stride=2)
        self.linear = nn.Linear(512 * block.expansion, num_classes)

    def _make_layer(self, block, planes, num_blocks, stride):
        strides = [stride] + [1] * (num_blocks - 1)
        layers = []
        for stride in strides:
            layers.append(block(self.in_planes, planes, stride))
            self.in_planes = planes * block.expansion
        return nn.Sequential(*layers)

    def forward(self, x):
        out = nn.ReLU()(self.bn1(self.conv1(x)))
        out = self.layer1(out)
        out = self.layer2(out)
        out = self.layer3(out)
        out = self.layer4(out)
        out = nn.AdaptiveAvgPool2d((1, 1))(out)
        out = out.view(out.size(0), -1)
        out = self.linear(out)
        return out

# Function to count parameters
def count_parameters(model):
    return sum(p.numel() for p in model.parameters())

# Function to estimate FLOPs
def get_flops(model, input_dim):
    model.eval()
    device = next(model.parameters()).device # Get the device of the model
    flops = torch.tensor([0.0], device=device)

    def add_hooks(module):
        if len(list(module.children())) > 0:
            return
        module.register_forward_hook(hook)

    def hook(module, input0, output):
        input0 = input0[0]
        output = output[0]

        batch_size = input0.shape[0]

```

```

        output_elements = output.numel()
        input_elements = input0.numel()

        flops_per_element = 2 # Assuming multiply-add operations
        if isinstance(module, (nn.Conv2d, nn.Linear)):
            flops_per_element = input_elements // batch_size

        flops.add_(batch_size * output_elements * flops_per_element)

    model.apply(add_hooks)

    dummy_input = torch.randn(input_dim)
    model(dummy_input.to(device))

    return int(flops.item())

# Models
pre_trained_model =
models.resnet152(weights=models.ResNet152_Weights.IMAGENET1K_V2)
transfer_learning_model = TransferLearningModel(pre_trained_model,
num_classes=20, dropout_rate=0.5, use_bn=True)
se_net = SEResNet(SEResNetBlock, [3, 8, 36, 3], num_classes=20)

# Count parameters
total_params_resnet152 = count_parameters(transfer_learning_model)
total_params_se_net = count_parameters(se_net)

# Estimate FLOPs
input_size = (1, 3, 224, 224) # BATCH_SIZE of 1
flops_resnet152 = get_flops(transfer_learning_model, input_size)
flops_se_net = get_flops(se_net, input_size)

print(f"ResNet - Total Parameters: {total_params_resnet152}, Estimated
Computations: {flops_resnet152}")
print(f"SeNet - Total Parameters: {total_params_se_net}, Estimated
Computations: {flops_se_net}")

```

5. Metric

It is important to note that the average weighted recall was used for recall calculation.

ResNet's performance metrics are given below:

Dropout	Batch Normalization	Epoch	Accuracy	Precision	Recall
0.0	False	121	0.975	0.975133335	0.975
0.0	True	087	0.964	0.964099568	0.964
0.2	False	138	0.975	0.975369150	0.975
0.2	True	087	0.966	0.966406526	0.966
0.3	False	127	0.974	0.974438370	0.974
0.3	True	090	0.971	0.971513341	0.971
0.4	False	117	0.969	0.969339367	0.969
0.4	True	108	0.967	0.967104320	0.967

0.5	False	156	0.973	0.973155326	0.973
0.5	True	095	0.965	0.965096725	0.965

SeNet's performance metrics are given below:

Dropout	Batch Normalization	Epoch	Accuracy	Precision	Recall
0.0	False	28	0.488	0.513466274	0.488
0.0	True	34	0.496	0.498935088	0.496
0.2	False	40	0.524	0.545607366	0.524
0.2	True	30	0.504	0.517430346	0.504
0.3	False	42	0.539	0.544515670	0.539
0.3	True	41	0.536	0.550808151	0.536
0.4	False	33	0.490	0.529336000	0.490

All the confusion matrices are given below:

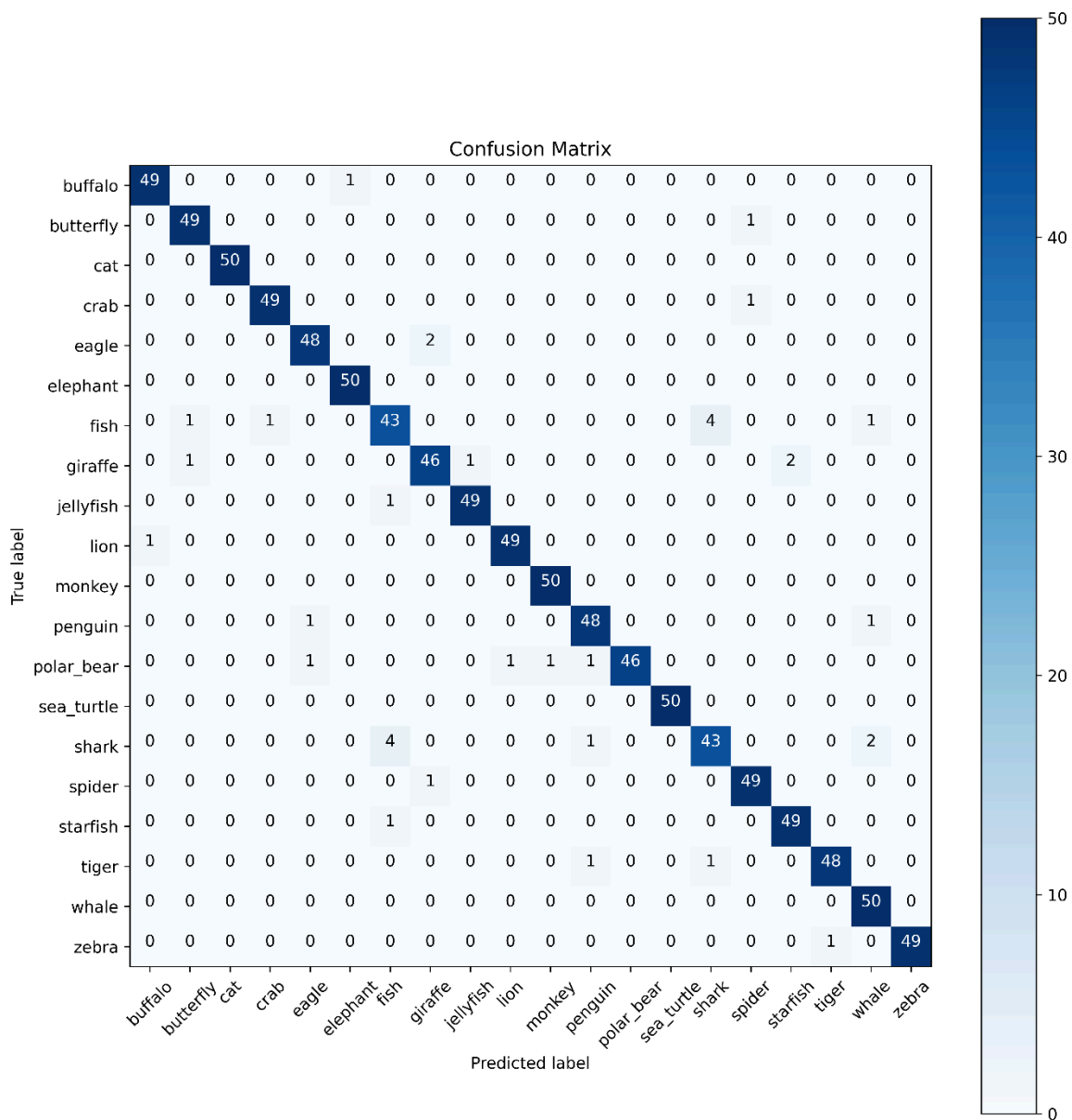


Figure 7: ResNet Dropout: 0.0, BN: False

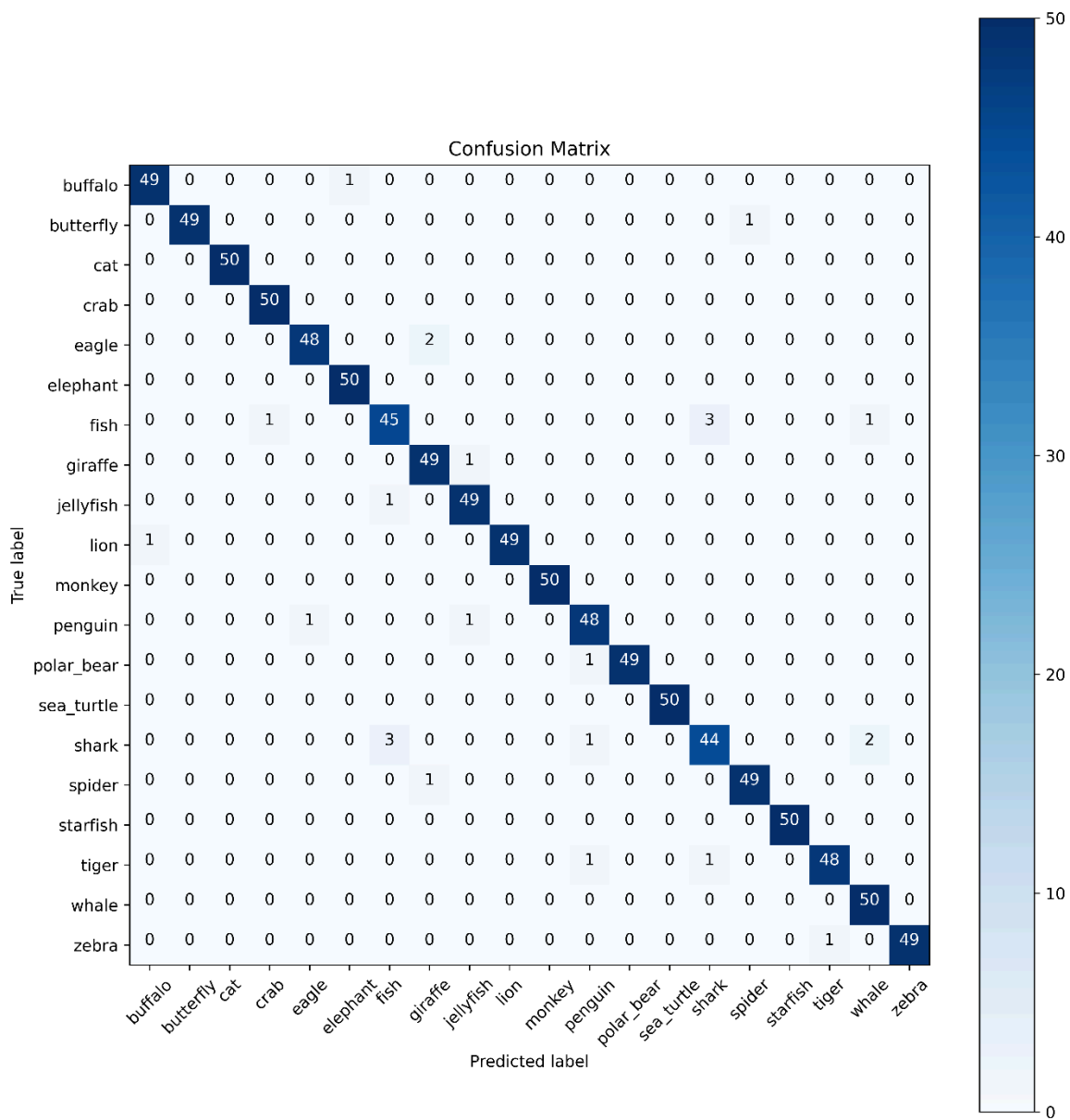


Figure 8: ResNet Dropout: 0.0, BN: True

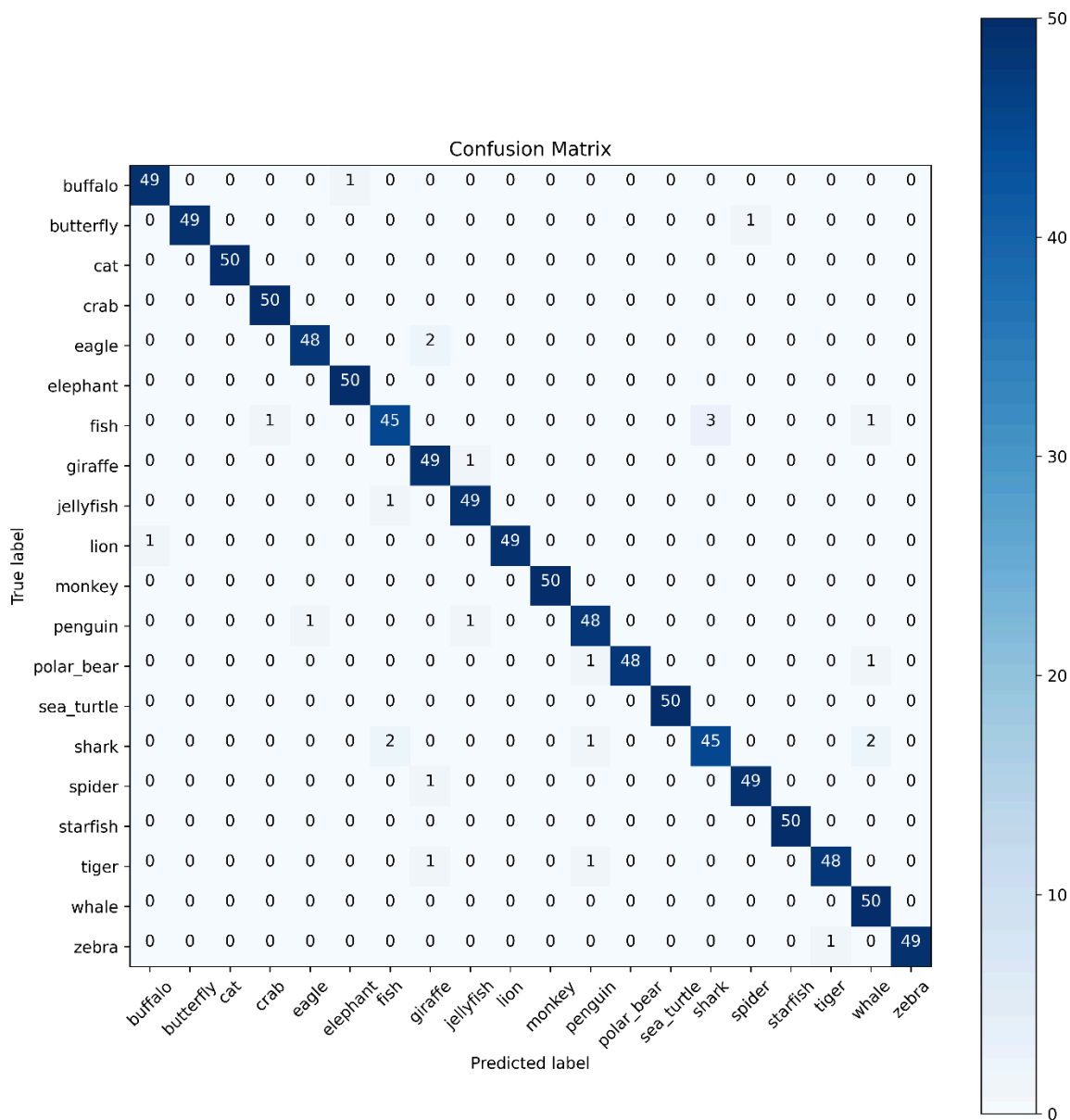


Figure 9: ResNet Dropout: 0.2, BN: False

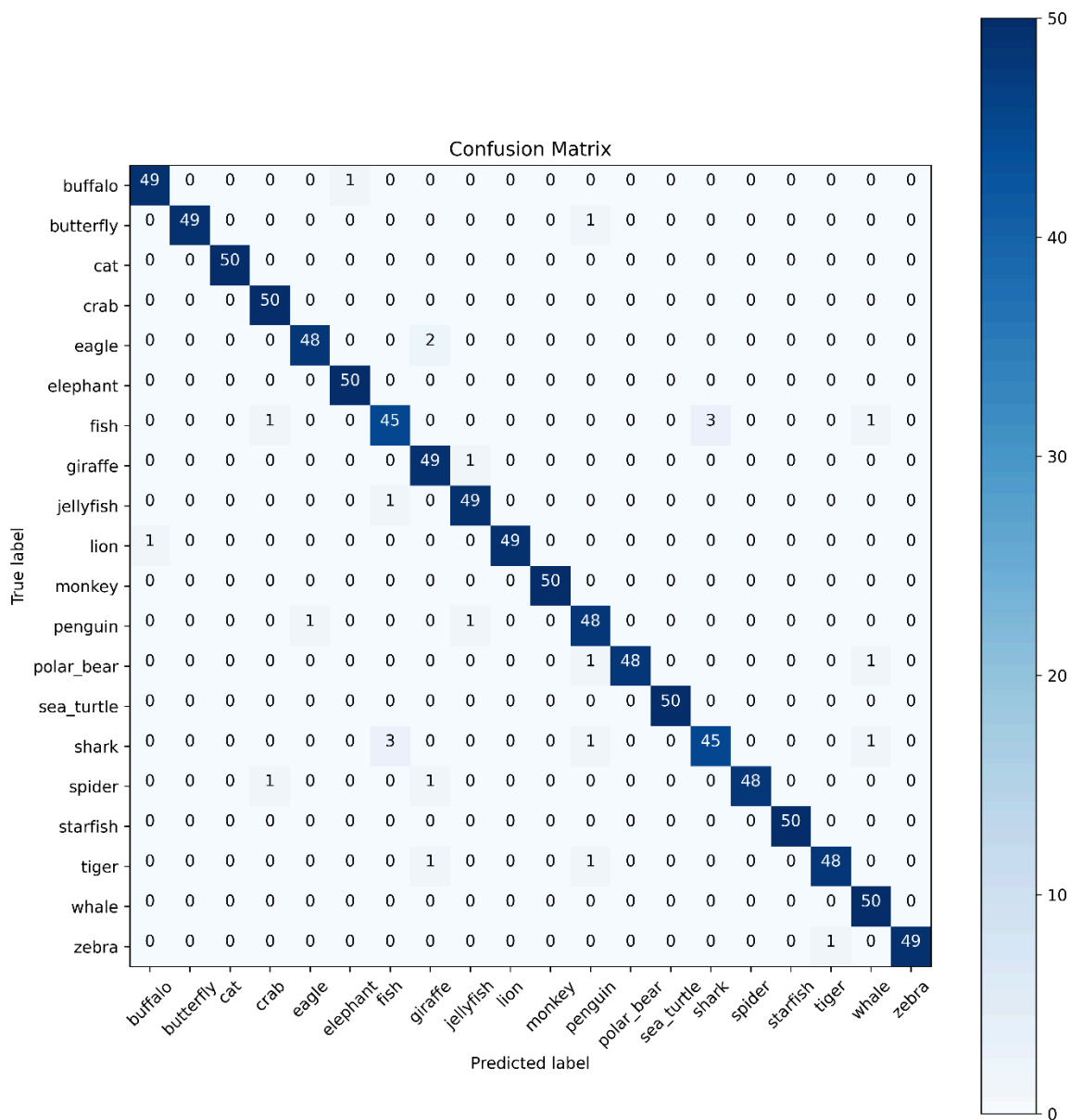


Figure 10: ResNet Dropout: 0.2, BN: True

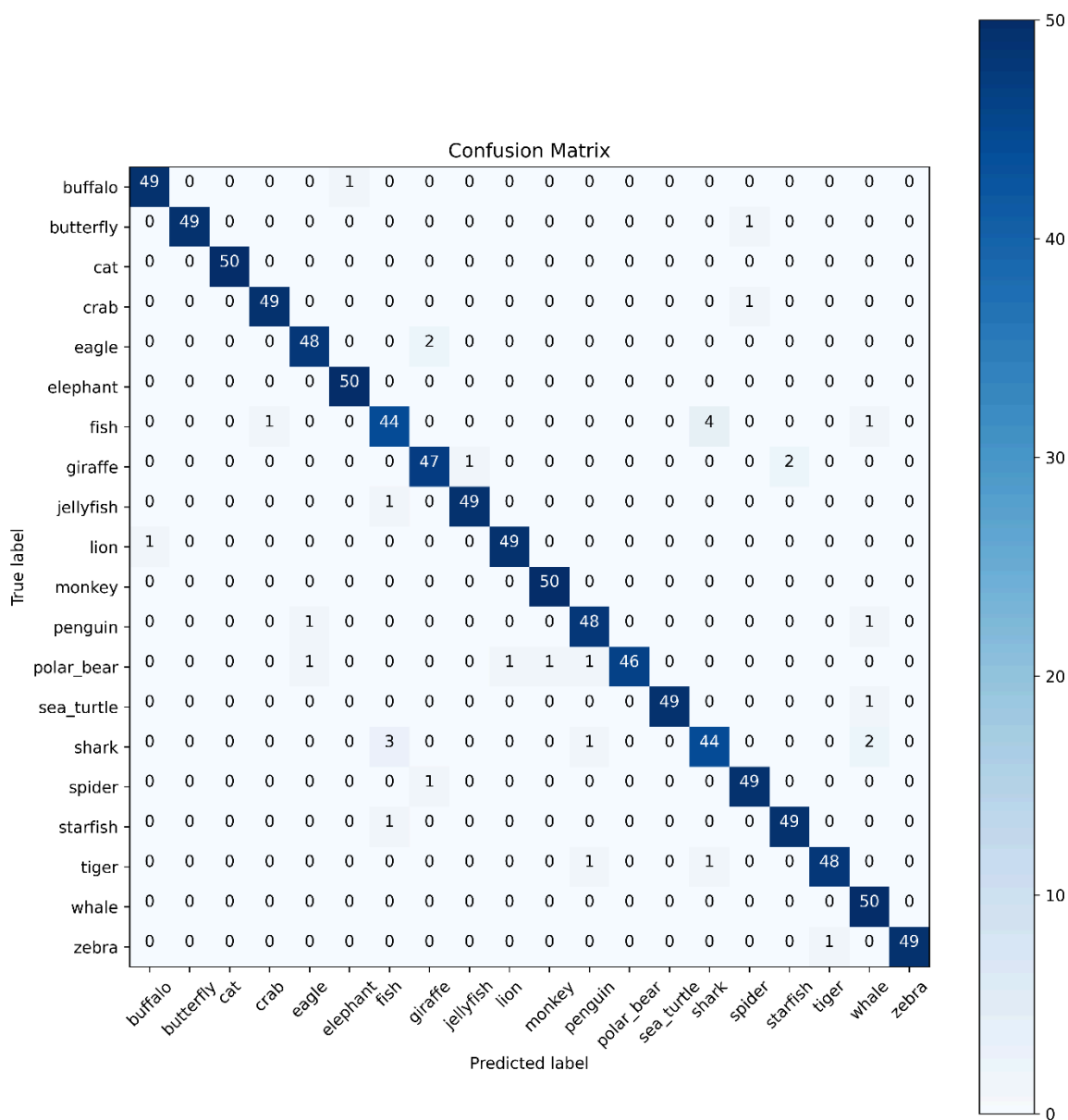


Figure 11: ResNet Dropout: 0.3, BN: False

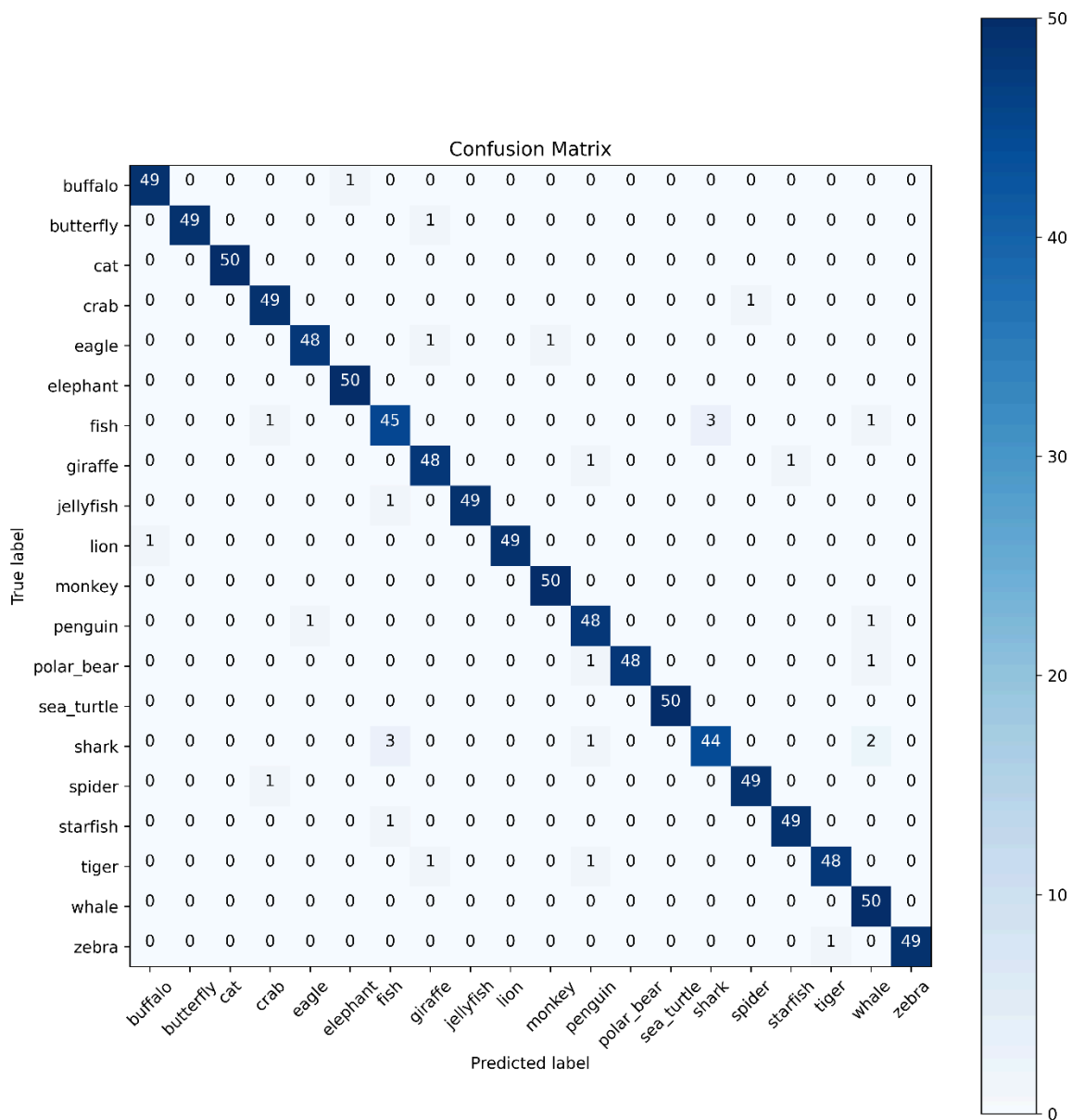


Figure 12: ResNet Dropout: 0.3, BN: True

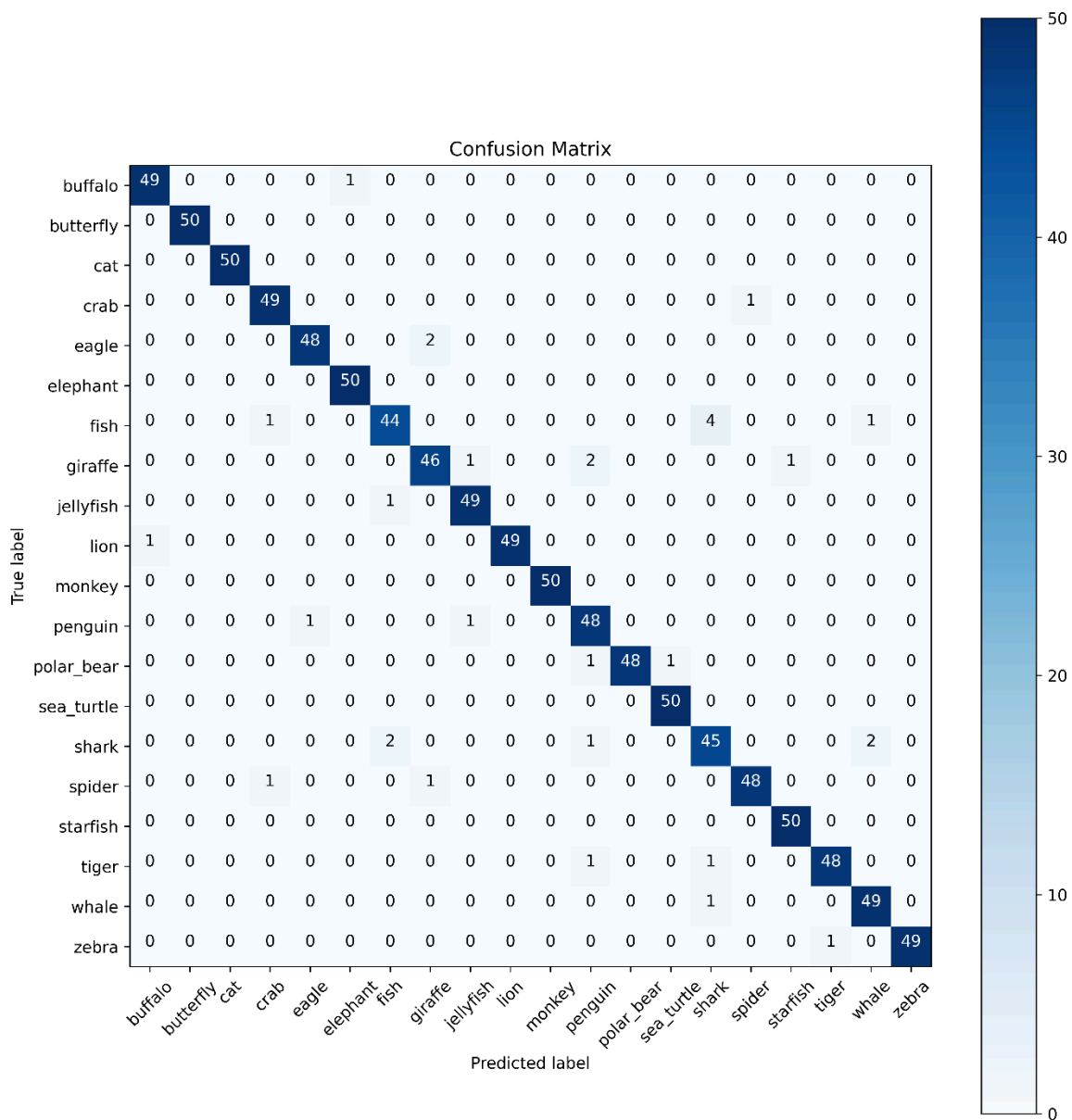


Figure 13: ResNet Dropout: 0.4, BN: False

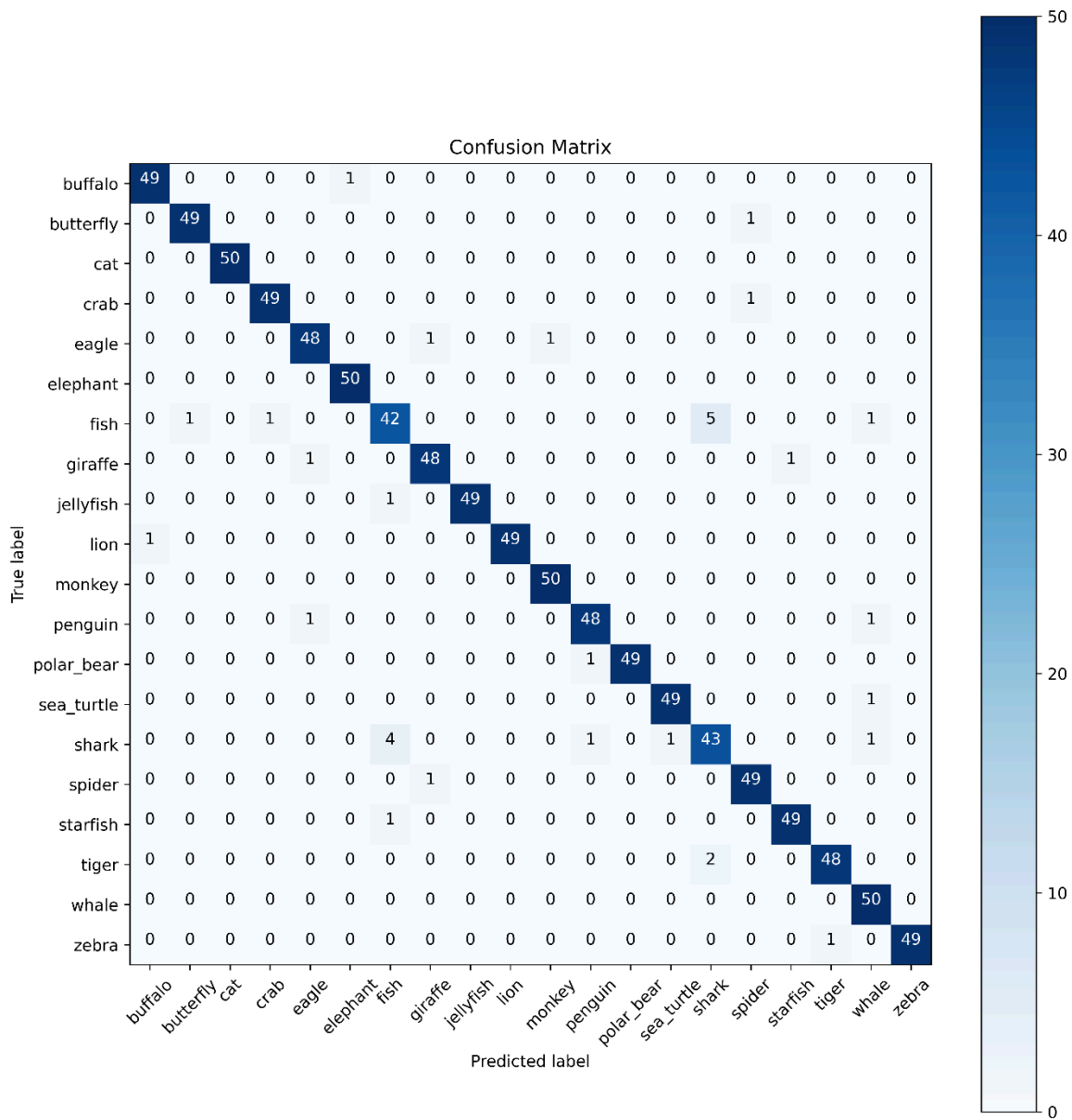


Figure 14: ResNet Dropout: 0.4, BN: True

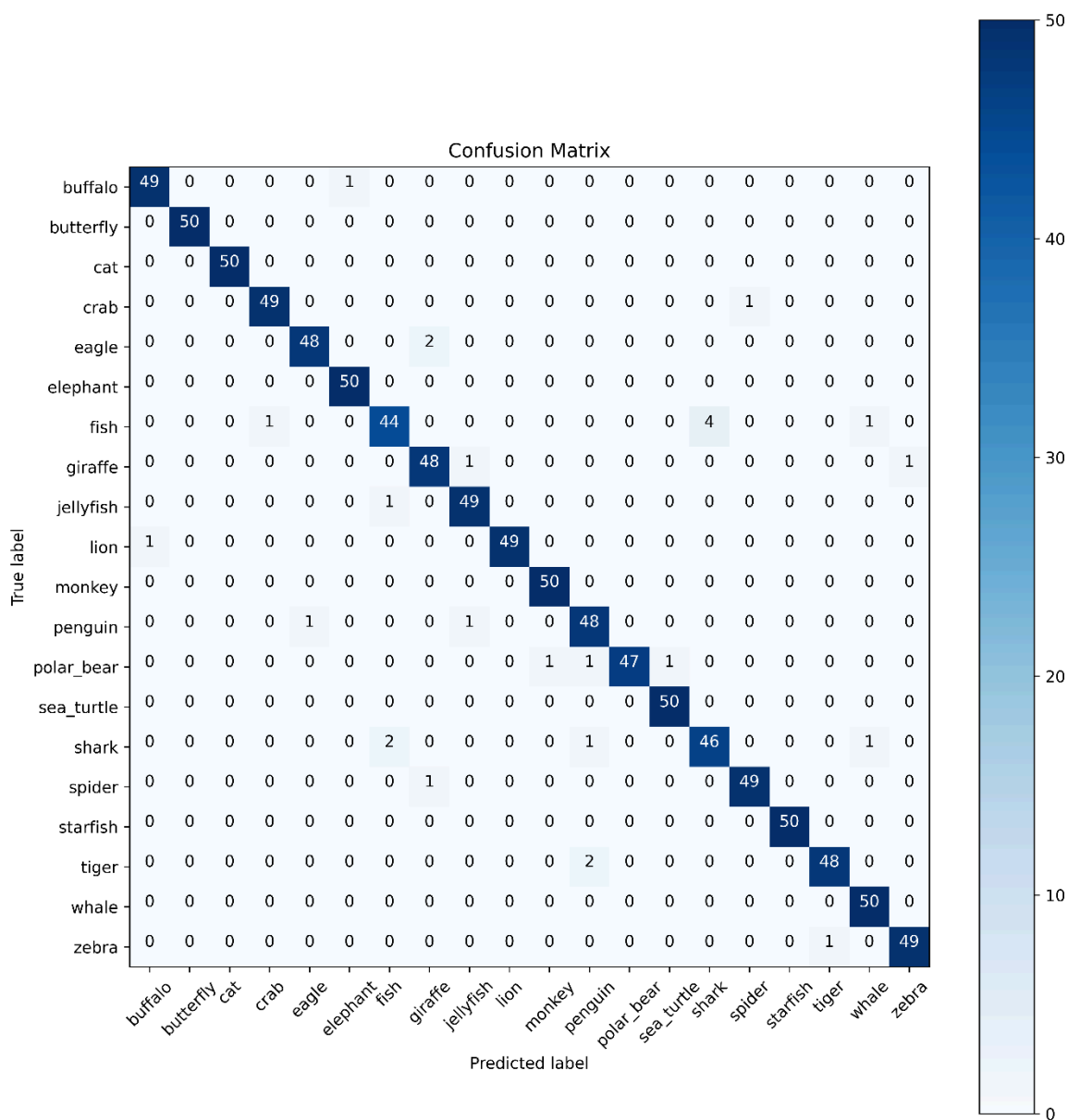


Figure 15: ResNet Dropout: 0.5, BN: False

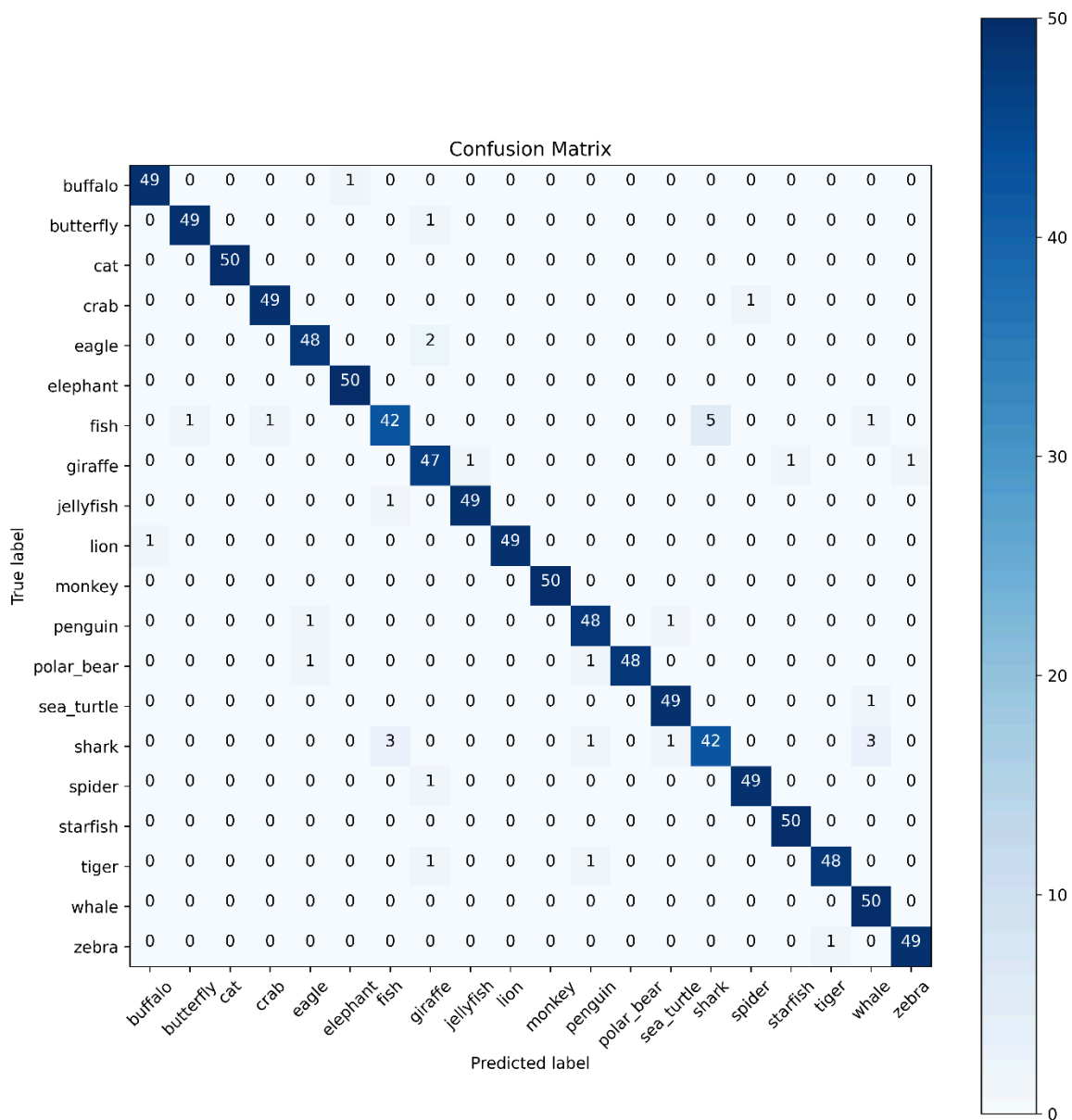


Figure 16: ResNet Dropout: 0.5, BN: True

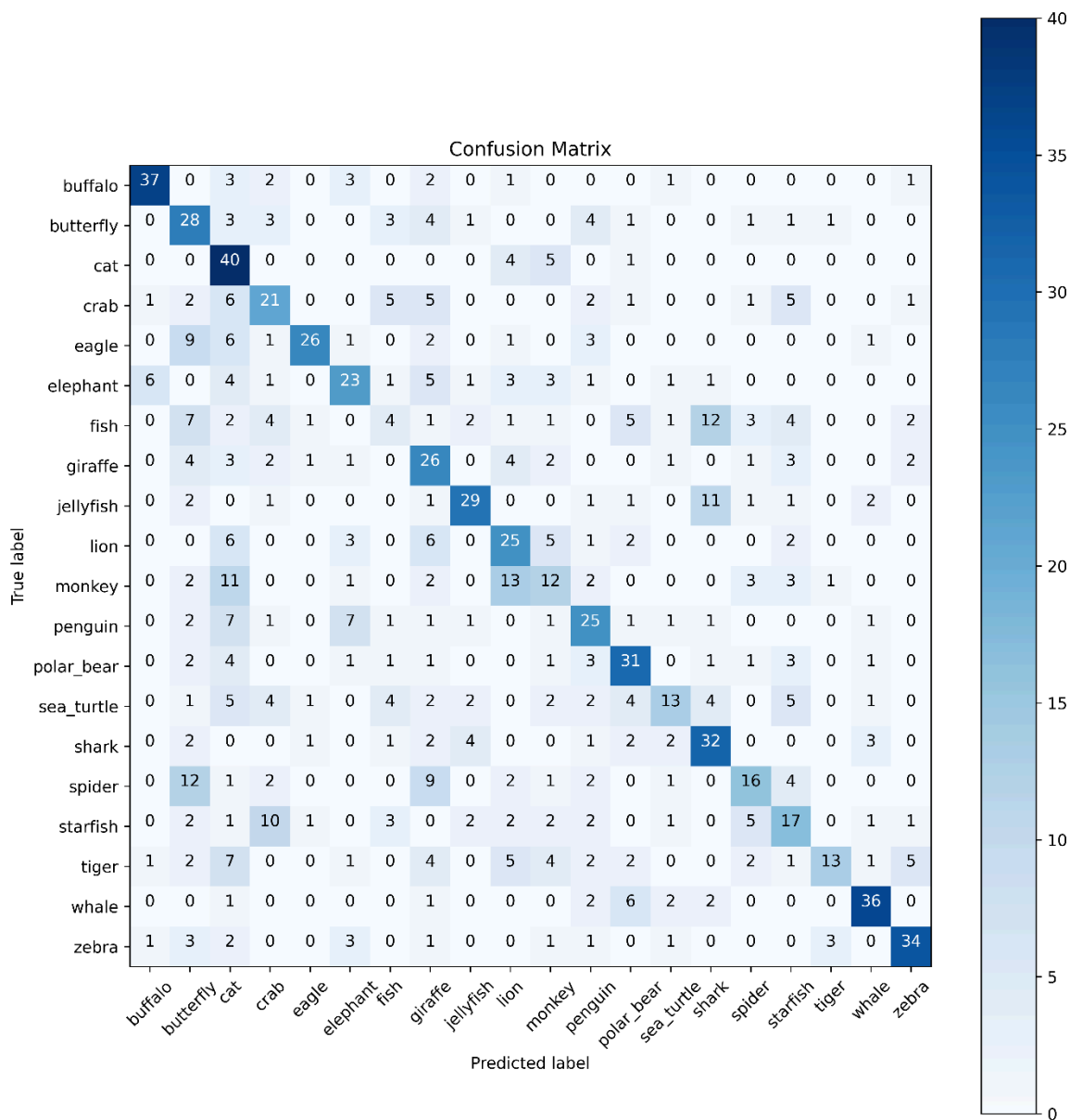


Figure 17: SeNet Dropout: 0.0, BN: False

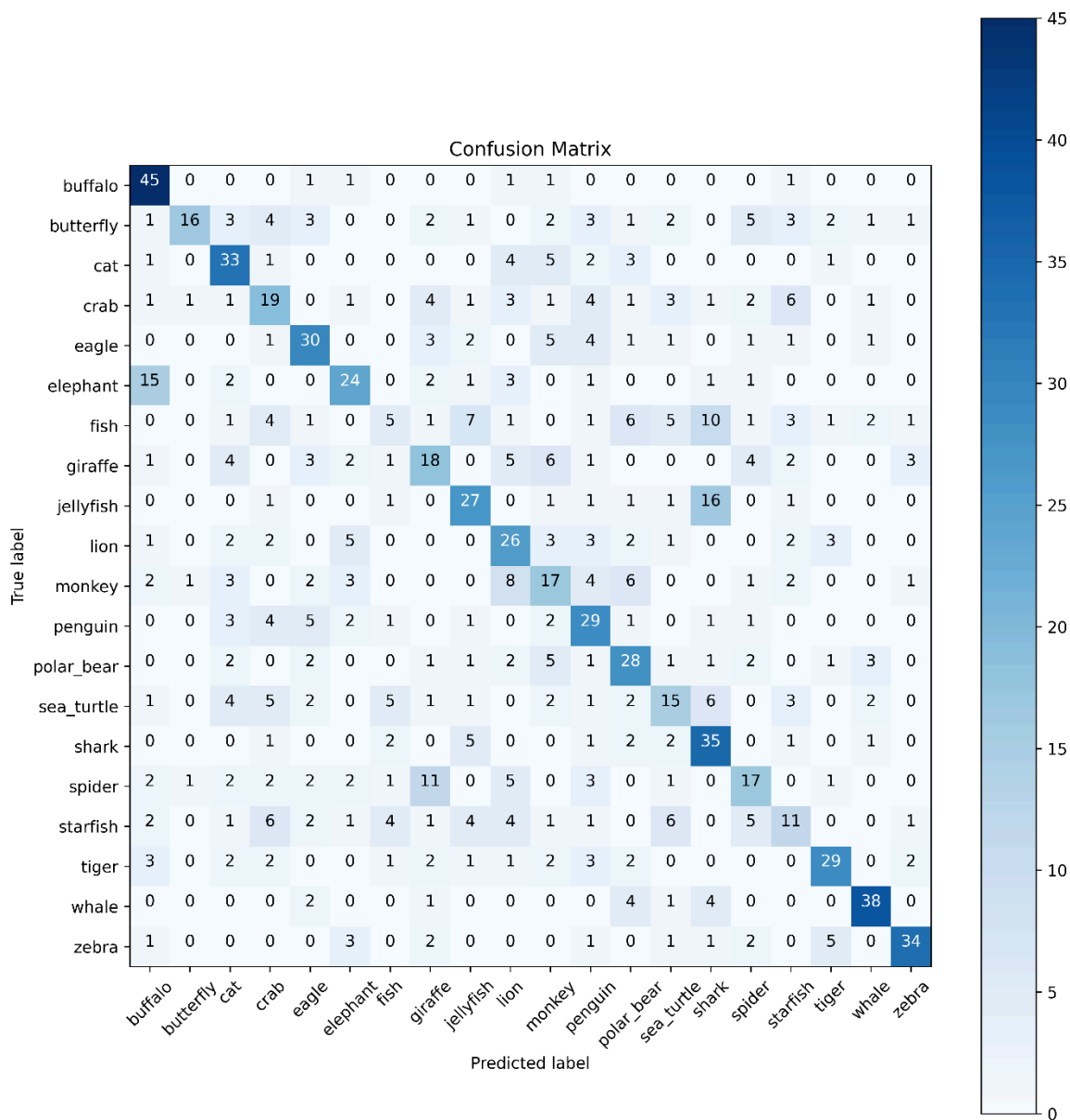


Figure 18: SeNet Dropout: 0.0, BN: True

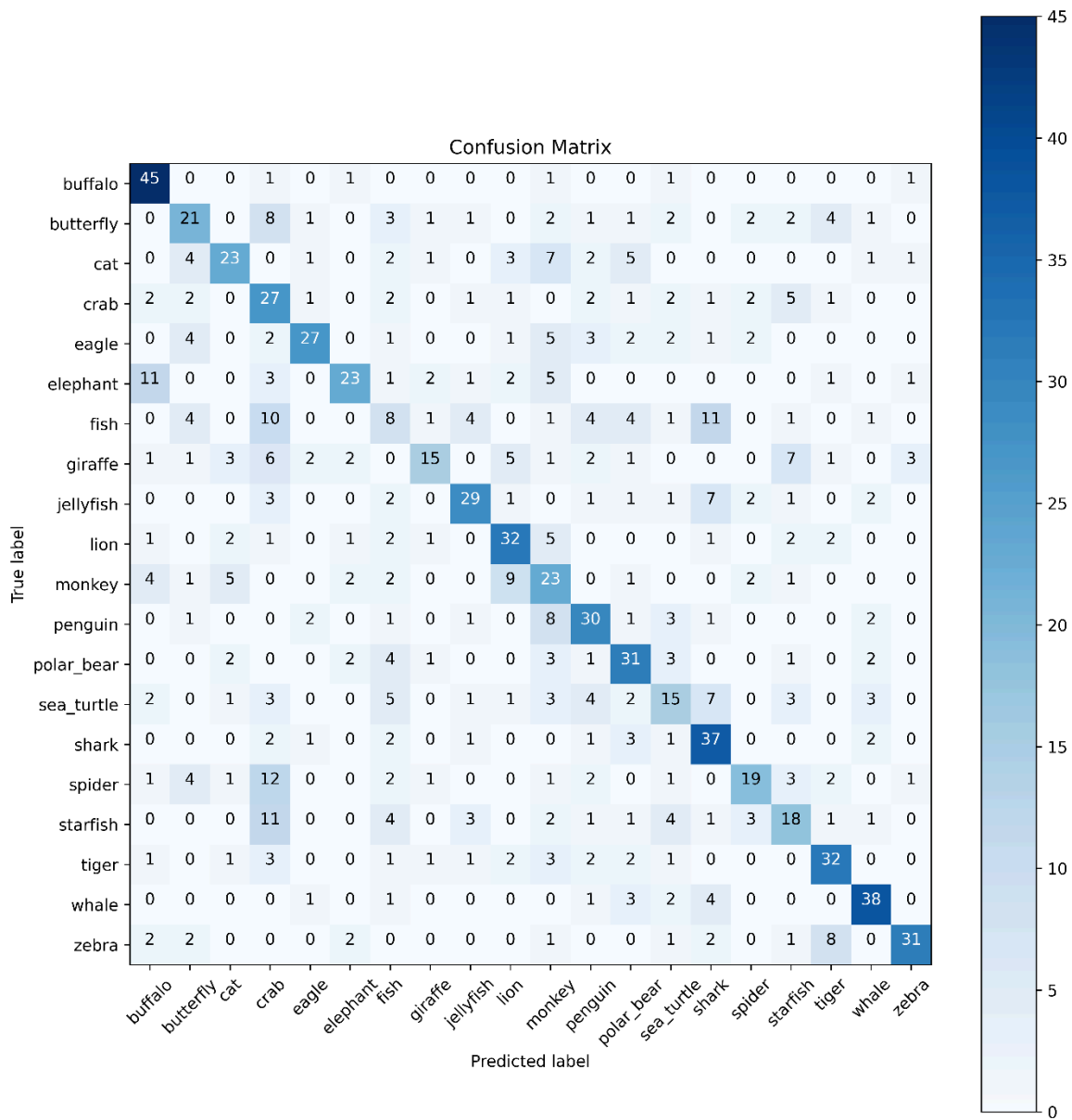


Figure 19: SeNet Dropout: 0.2, BN: False

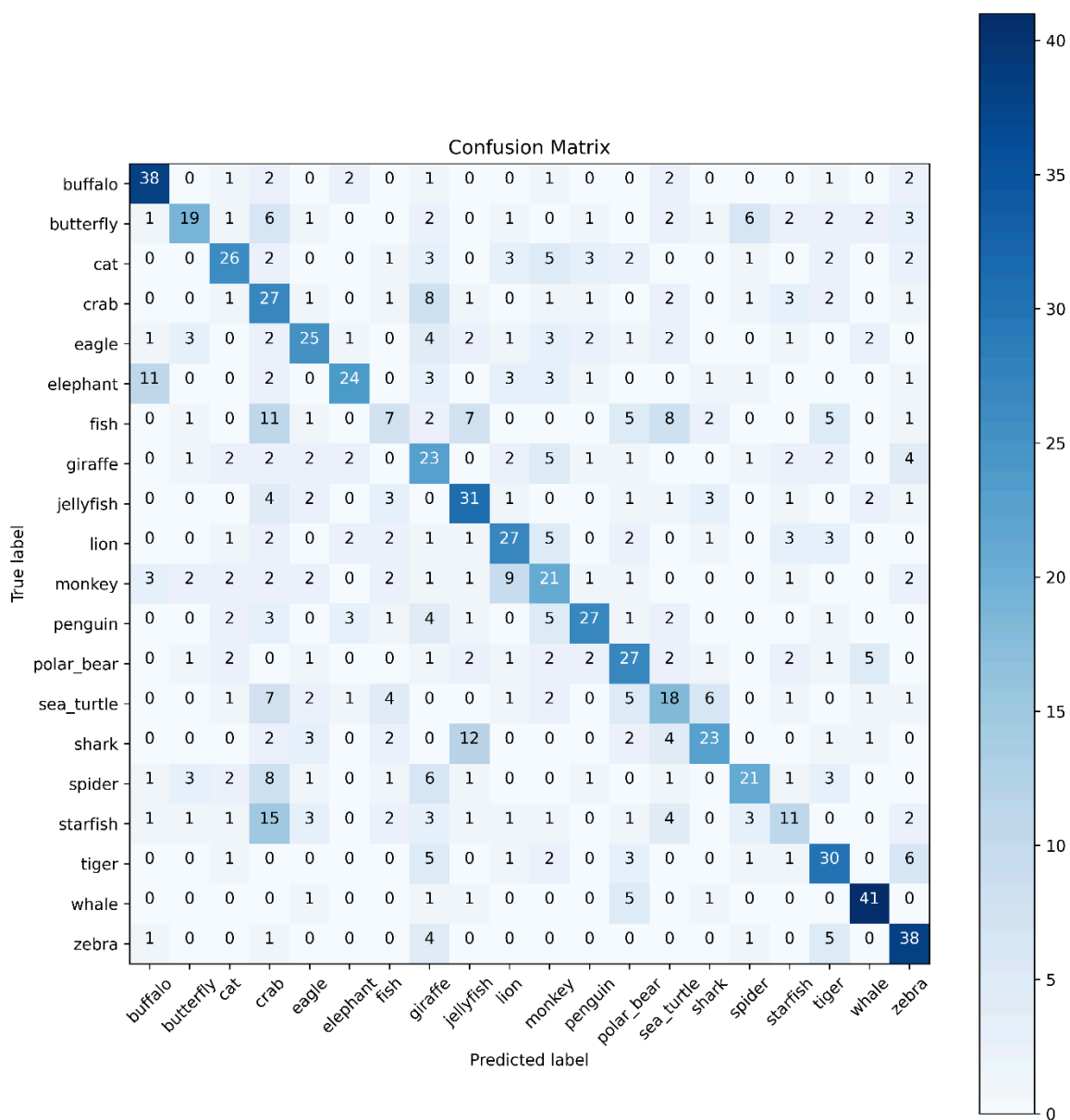


Figure 20: SeNet Dropout: 0.2, BN: True

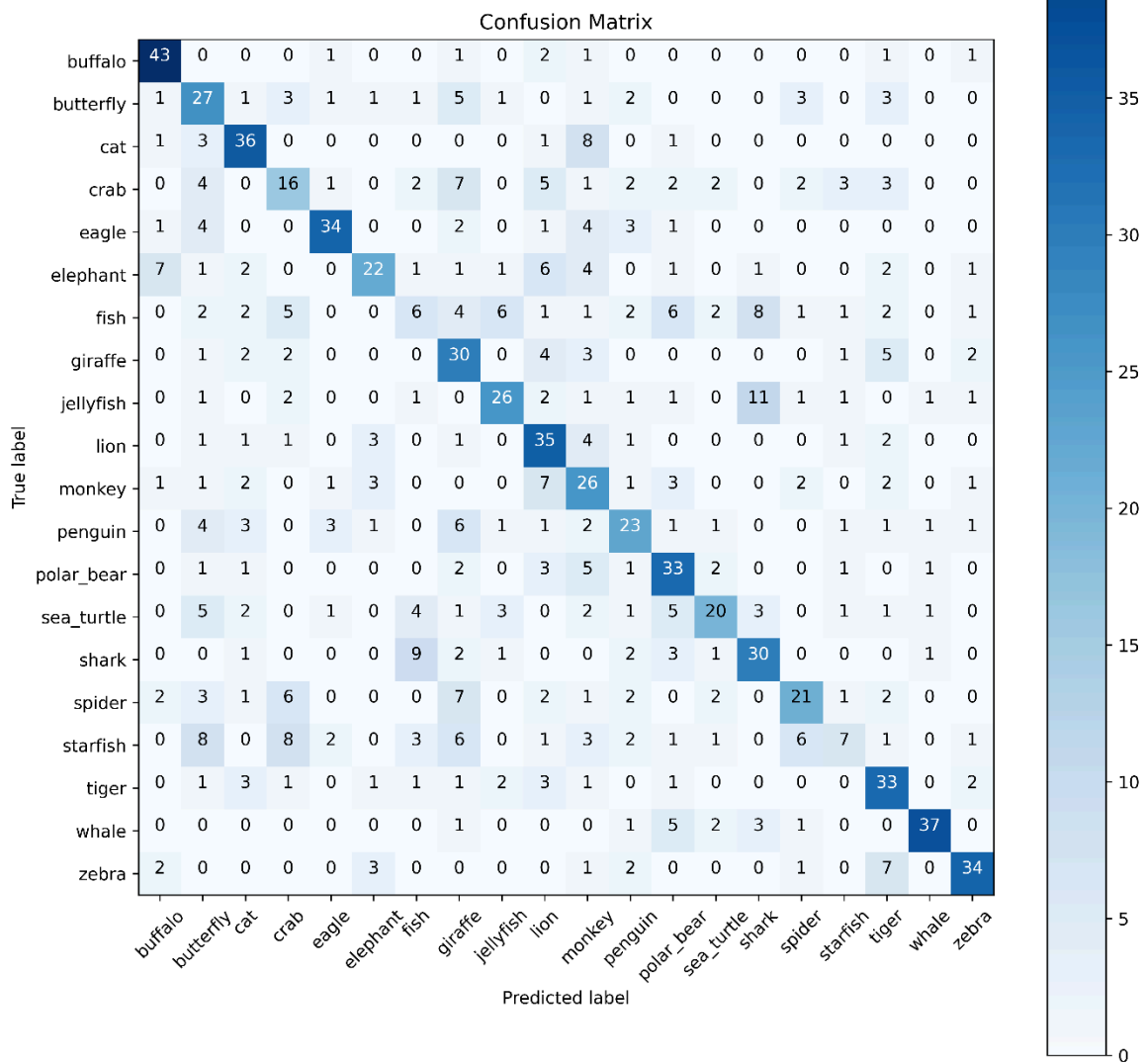


Figure 21: SeNet Dropout: 0.3, BN: False

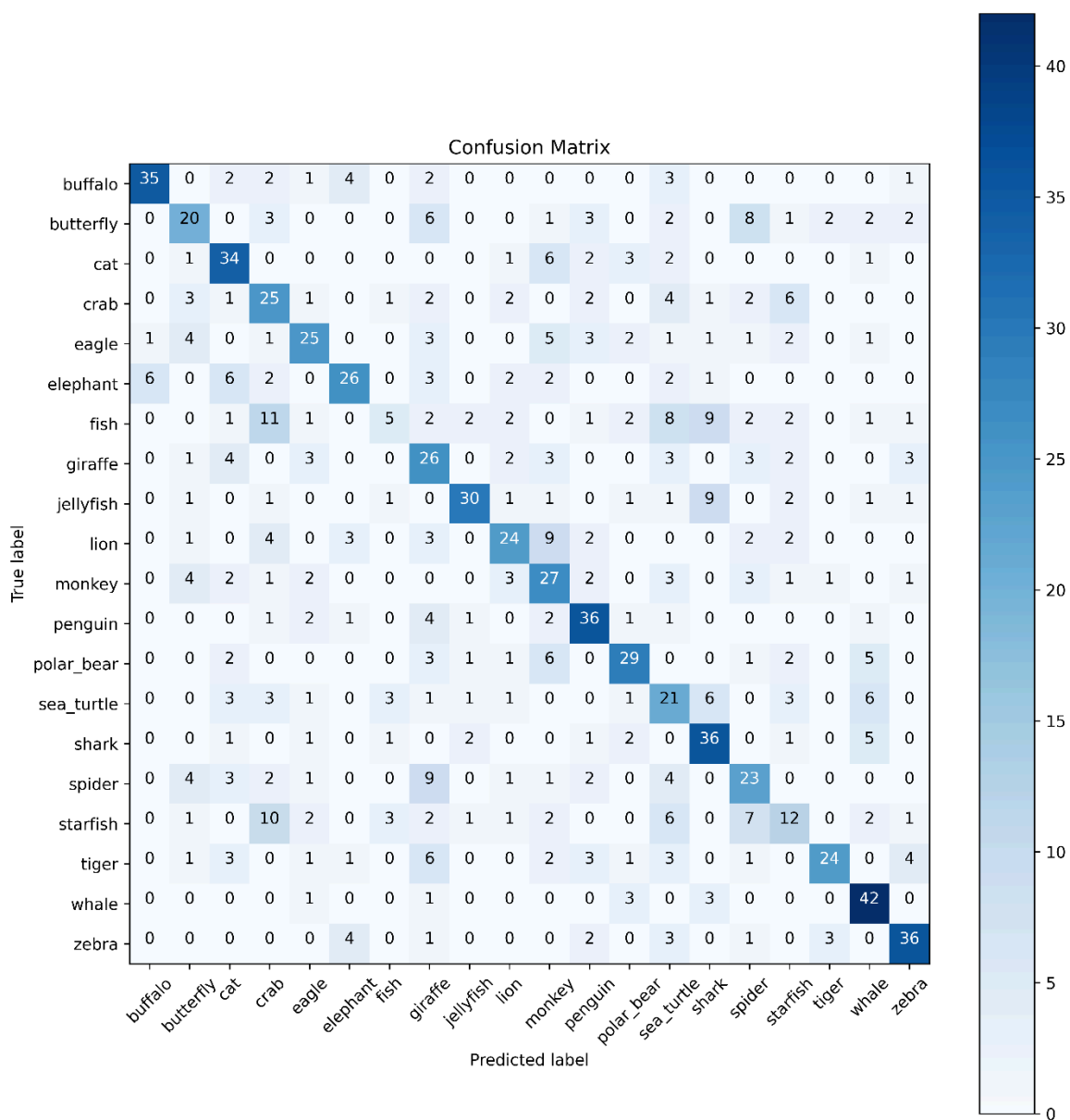


Figure 22: SeNet Dropout: 0.3, BN: True

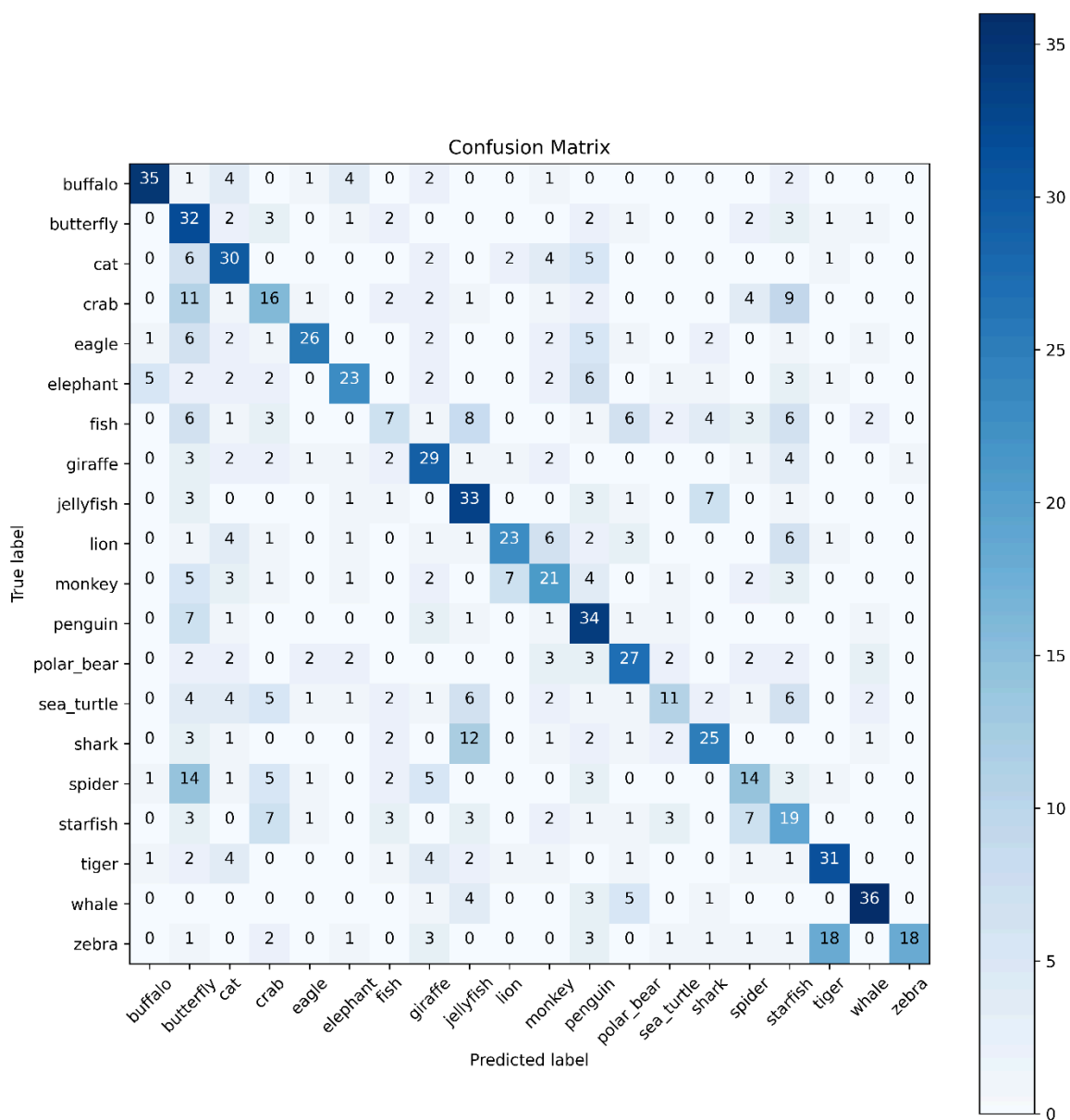


Figure 23: SeNet Dropout: 0.4, BN: False